

DOCUMENTACIÓN SASS-ISO-II

Sergio Delgado Sánchez

Adrián Bohabén Torés

Sergio Fernández Del Río

Samantha Micaela Velasco Pinta

Resumen

En este documento se presenta el desarrollo de una aplicación web de reserva de alojamientos entre particulares realizada en el marco de la asignatura Ingeniería del Software II. El proyecto aborda el ciclo completo de desarrollo de un sistema software, desde la definición de requisitos hasta la implementación y validación del producto final.

El sistema permite la interacción entre distintos tipos de usuarios, posibilitando la publicación de alojamientos y la gestión de reservas a través de la plataforma. El trabajo integra prácticas habituales del desarrollo profesional, incorporando control de versiones, pruebas automatizadas y mecanismos de aseguramiento de la calidad. El documento recoge el resultado final del proyecto y las principales decisiones técnicas adoptadas durante su desarrollo.

Índice

Resumen	1
Introducción	4
Organización del proyecto	5
Datos de la empresa	5
Metodología de desarrollo.....	6
Planificación	7
Backlog	8
Diseño del Sistema.....	12
Especificación.....	12
Análisis	13
Actores	13
Casos de uso.....	13
Diagrama	14
Diseño	14
Capas	14
Diagrama	15
Diagrama	16
Funcionalidades	17
Usuario	17
Propietario	17
Inquilino	17
Implementación	18
Requisitos previos	18
Tecnologías utilizadas	18
Estructura del proyecto	19
Calidad del software	20
Análisis y Configuración de Reglas	20
Inyección de Dependencias	20
Refactorización y Corrección de "Code Smells"	20
Estado Final y Últimas Correcciones	21

Pruebas	22
Gestor Inmuebles.....	23
LNBusqueda	24
Gestor Inicio	25
Gestor Lista de Deseos.....	25
Gestor Notificaciones.....	26
Gestor Pagos	26
Gestor Reservas	27
Gestor Usuarios.....	27
InmueblesDAO	28
InquilinoDAO.....	29
PropietarioDAO	30
ReservaDAO	30
SolicitudReservaDAO	32
UsuarioDAO.....	33
Guía de instalación y ejecución.....	35
Pasos previos.....	35
Configuración del entorno	35
Ejecución	35
Demostración del sistema.....	36
Mantenimiento y evolución.....	43
Mantenimiento correctivo.....	43
Mantenimiento evolutivo	43
Mantenimiento adaptativo	44
Conclusiones	44

Introducción

Este proyecto se desarrolla en el marco de una empresa ficticia de desarrollo software, denominada **SASS**, formada por cuatro desarrolladores, organizada de manera autogestionada y colaborativa. Dentro del equipo se definen roles claros, incluyendo la figura del *Product Owner*, responsable de la comunicación externa, así como otros roles orientados a la gestión del producto y al desarrollo técnico.

El objetivo técnico del proyecto es el desarrollo de una aplicación web de reserva de alojamientos entre particulares, inspirada en plataformas existentes de este tipo. Desde el punto de vista metodológico, el proyecto se desarrolla siguiendo un enfoque iterativo e incremental, utilizando el concepto de versiones y *sprints* para planificar el trabajo y monitorizar su evolución. A su vez, se incorporan mecanismos de análisis estático, pruebas automatizadas y validación funcional.

El presente documento describe de forma estructurada todo este proceso, abordando la organización del proyecto, la metodología de desarrollo empleada, la especificación de requisitos, el análisis y diseño del sistema, su implementación, las estrategias de calidad y pruebas aplicadas, así como los planes de mantenimiento y conclusiones derivadas del trabajo realizado.

Organización del proyecto

Datos de la empresa

- I. Nombre. SASS, es el acrónimo de las iniciales de los nombres que conforman el equipo de desarrollo: Samantha, Adrián, Sergio y Sergio.
- II. Miembros del equipo:
 - a. Adrián Bohabén Torés
 - b. Samantha Micaela Velasco Pinta
 - c. Sergio Delgado Sánchez
 - d. Sergio Fernández Del Río
- III. Roles:
 - a. *Product Owner*. Adrián Bohabén Torés.
 - b. *Scrum Master*. Adrián Bohabén Torés.
 - c. *Software Development Manager*. Samantha Micaela Velasco Pinta.
 - d. *Fullstack Developer*. Adrián Bohabén Torés, Samantha Micaela Velasco Pinta, Sergio Delgado Sánchez y Sergio Fernández Del Río.
- IV. Modalidad de trabajo. Híbrida, el grupo de desarrollo trabaja telemática y presencialmente.
 - a. Reuniones. El grupo de desarrollo se reúne los lunes de forma presencial en las instalaciones proporcionadas por la universidad en el transcurso de la hora de laboratorio de la asignatura. Esta reunión tiene como objetivo consultar dudas con el profesorado y avanzar en el desarrollo lo que sea posible. Además, cada viernes se realiza un *sprint review* para poner en común los avances de la semana, y establecer los objetivos de la siguiente.
 - b. Comunicación con el profesorado. El Product Owner es el encargado de la externalización de la comunicación con el profesorado. Esta comunicación se lleva a cabo de forma presencial durante las horas lectivas de la asignatura, y telemáticamente a través de correo electrónico.

Metodología de desarrollo

- I. Metodología utilizada. Se ha utilizado una metodología ágil inspirada en Scrum, caracterizada por el desarrollo iterativo e incremental del producto, la entrega de un producto mínimo viable y la adaptación continua de las herramientas proporcionadas por el profesorado para su desarrollo.
- II. Planificación.
 - a. Reuniones. El grupo de desarrollo se reúne los lunes de forma presencial en las instalaciones proporcionadas por la universidad en el transcurso de la hora de laboratorio de la asignatura. Esta reunión tiene como objetivo consultar dudas con el profesorado y avanzar en el desarrollo lo que sea posible. Además, cada viernes se realiza un *sprint review* para poner en común los avances de la semana, y establecer los objetivos de la siguiente.
 - b. Comunicación con el profesorado. El *Product Owner* es el encargado de la externalización de la comunicación con el profesorado. Esta comunicación se lleva a cabo de forma presencial durante las horas lectivas de la asignatura, y telemáticamente a través de correo electrónico.
- III. Uso de *sprints*. El desarrollo se organizó en sprints semanales, en los que se planificaban las tareas prioritarias, se desarrollaban nuevas funcionalidades y se revisaban los avances al final de cada iteración.
- IV. Gestión de versiones. El sistema se ha desarrollado mediante versiones incrementales, comenzando por un MVP y ampliando progresivamente hasta su versión final. Para ello hemos hecho uso de GitHub.
 - a. Main. Contiene la versión estable del proyecto en producción.
 - b. Development. Contiene la última versión estable en desarrollo.
 - c. Reléase. Contiene la versión estable del proyecto en preproducción.
 - d. Hotfix. Contiene la versión que hay en producción tras detectar un error crítico.
 - e. Feature.n. Cada una de las “n” ramas contiene el desarrollo de una nueva funcionalidad.
- V. Autogestión. Durante el desarrollo, usamos herramientas integradas en GitHub para el manejo de actividades e incidencias.
 - a. Issues. Permiten registrar y gestionar tareas, mejoras y errores detectados durante el desarrollo. Cada *issue* representa una unidad de trabajo concreta.
 - b. Milestones. Permiten controlar el avance global del proyecto, evaluar el grado de cumplimiento de los objetivos planificados y verificar qué tareas han sido completadas dentro de cada iteración.
 - c. Project Board. Permite el seguimiento visual del proyecto, basado en un flujo tipo Kanban.

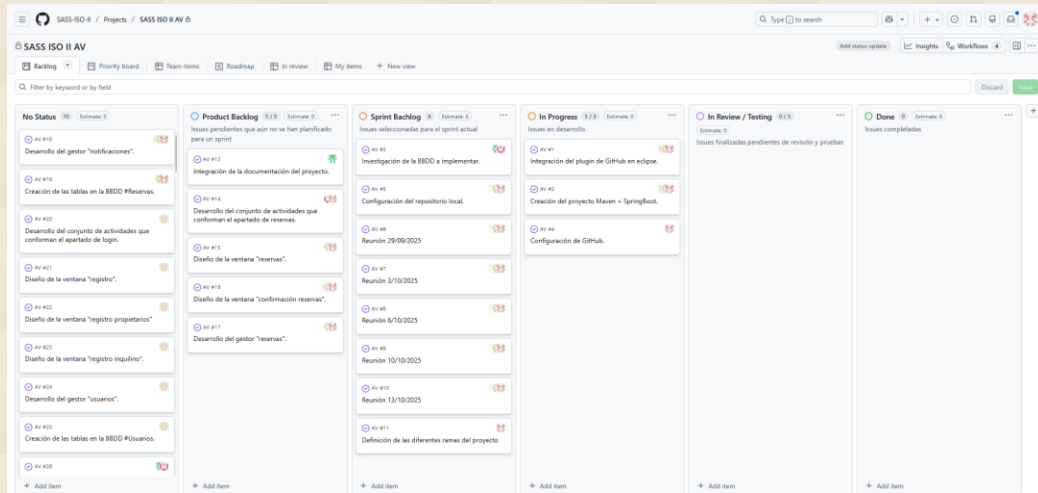
Planificación

El proyecto se planificó en diez *sprints*, cada uno con una duración de una semana. Debido a problemas que fueron surgiendo durante el desarrollo, no pudimos seguir el ritmo propuesto para los *sprints*, por lo que muchas *issues* se cerraron con un considerable retraso.

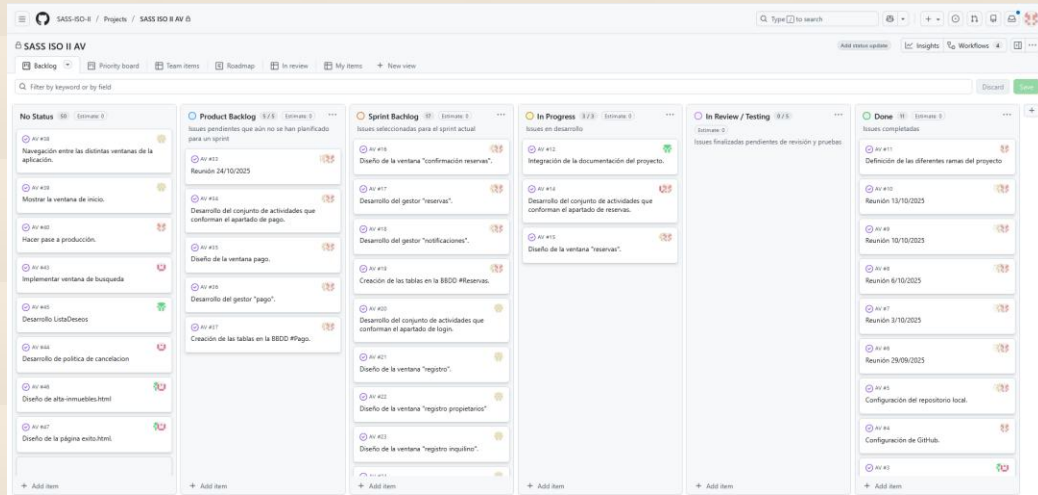
#Sprint	Duración	Objetivo	Funcionalidades
1	14 días	Configuración del entorno de trabajo: GitHub, Visual Paradigm, Beaver, ..., etc.	En este primer sprint no desarrollamos funcionalidades del proyecto
2	14 días	Inicio del MPV	Registro e inicio de sesión Alta de inmuebles Reserva de inmuebles
3	14 días	Finalización del MPV	Función de pago Página de inicio Navegación entre páginas
4	7 días	Integración de SonarQube Desarrollar las funcionalidades restantes del proyecto.	Política de cancelación Lista de Deseos Búsqueda de inmuebles Confirmación reservas Notificaciones
5	7 días	Corrección de bugs en las distintas capas del proyecto Diseño de las ventanas HTML	Eliminar ventanas innecesarias y añadir pop-ups que las sustituyan Campo fecha en ventana inmuebles
6	7 días	Integración del análisis en SonarQube	Configuración de <i>Quality Gate</i> Configuración de <i>Quality Profile</i>
7	7 días	Corrección de Issues marcadas por SonarQube	En este sprint se corrigen apartados como <i>code smell</i>
8	7 días	Integración de JUnit y elaboración de un plan de pruebas	En este Sprint se ha elaborado un plan de pruebas con el uso de Junit
9	7 días	Integración del entorno de pruebas SureFire	En este Sprint se ha integrado el plugin de Surefire para ejecutar las pruebas unitarias automáticamente
10	14 días	Preparación del producto final.	En este sprint no se implementan funcionalidades, se prepara el producto para su presentación

Backlog

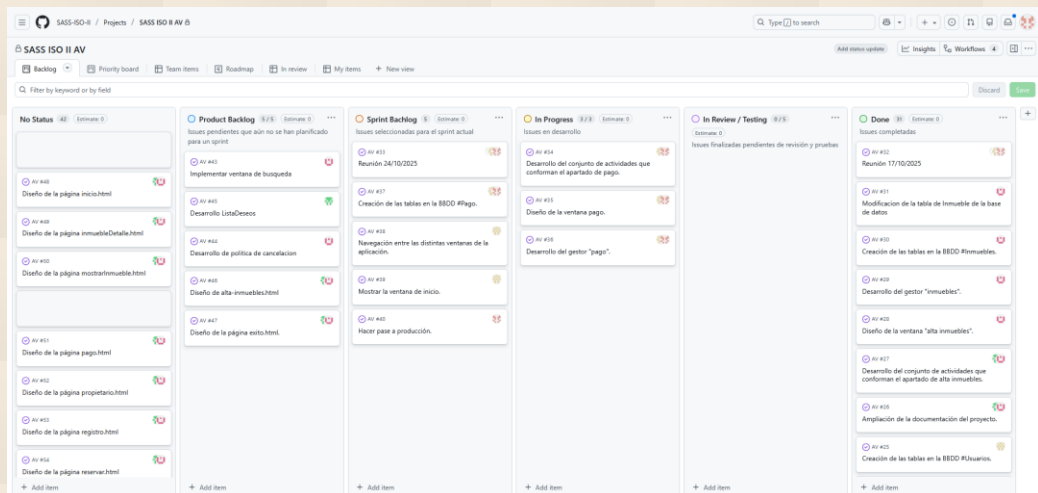
I. Sprint



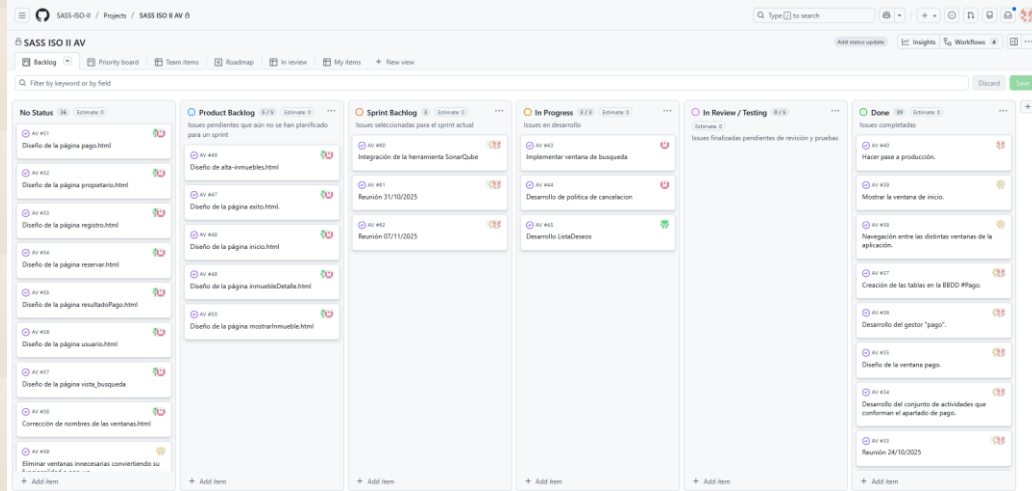
II. Sprint



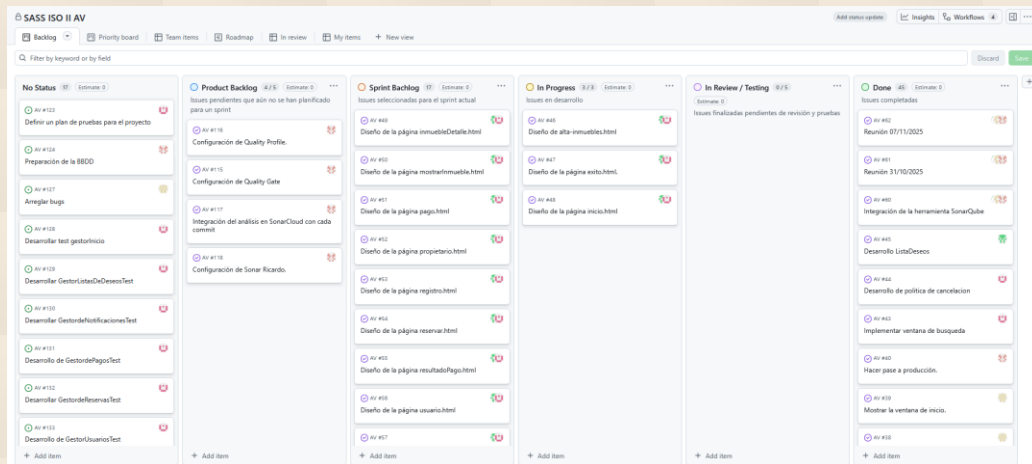
III. Sprint



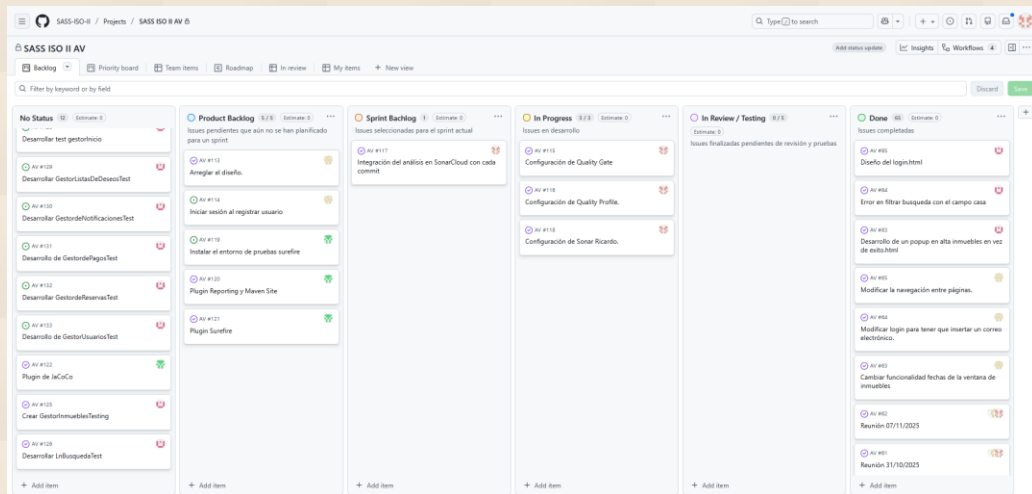
IV. Sprint



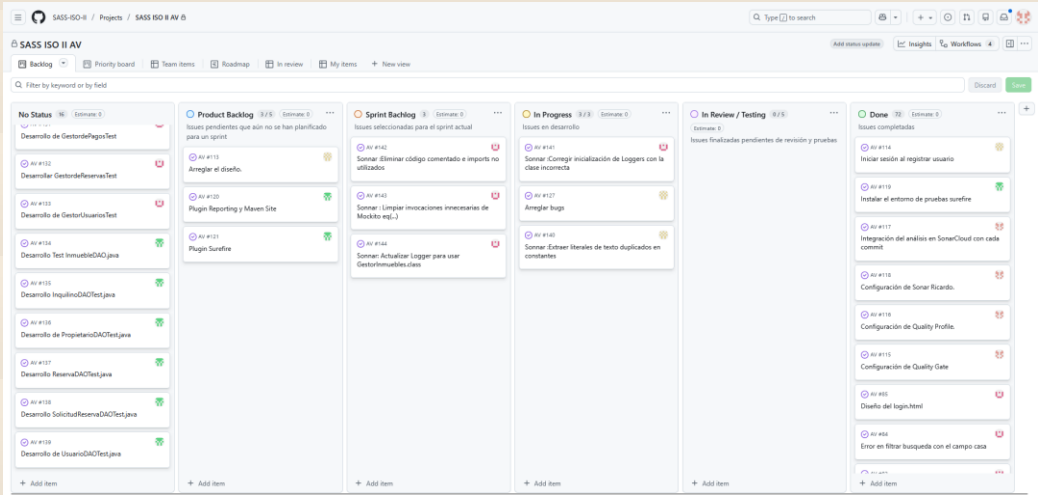
V. Sprint



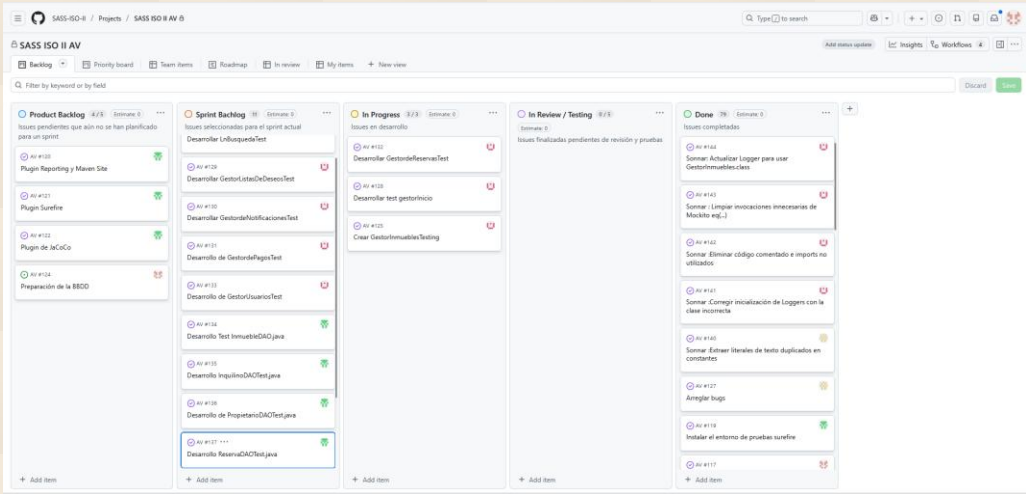
VI. Sprint



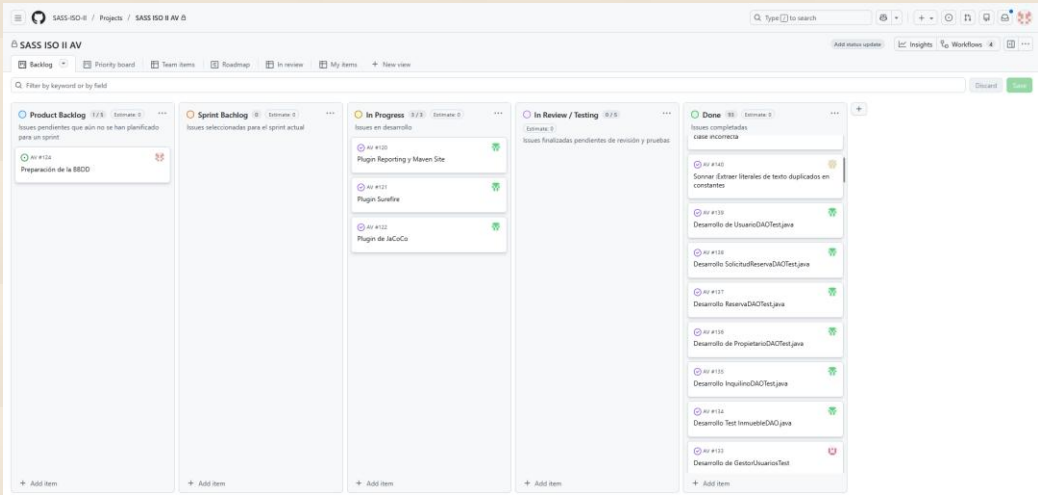
VII. Sprint



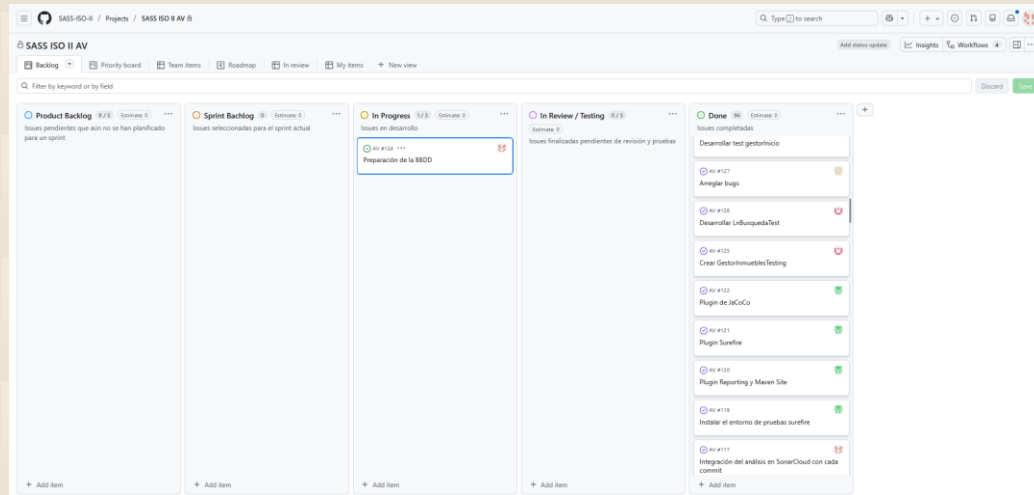
VIII. Sprint



IX. Sprint



X. Sprint



Diseño del Sistema

Especificación

Se pretende desarrollar un sistema similar a Airbnb donde personas particulares pueden reservar alojamientos de propietarios particulares. Tanto los inquilinos como propietarios deben registrarse previamente para utilizar el sistema.

- I. Los propietarios, tras iniciar sesión, pueden dar de alta propiedades ofertándolas para ser alquiladas. Los propietarios marcan si desean que los usuarios realicen reservas directas, o guardarse el derecho de confirmar o no las solicitudes de reserva.
- II. Los usuarios pueden primero buscar alojamiento para examinar las opciones de alquiler. Excepcionalmente también pueden aplicar filtros avanzados para, por ejemplo: ver aquellos inmuebles con posibilidad de reserva inmediata; seleccionar filtros para ciertas comodidades; o seleccionar política de cancelación de reserva.
- III. Para realizar la búsqueda de inmuebles no es necesario haber iniciado sesión, pero los usuarios registrados pueden agregar algunas propiedades a su lista de deseo.
- IV. Los inquilinos, tras iniciar sesión, una vez han seleccionado el inmueble a alquilar, pueden completar la reserva. Si la propiedad permite reserva inmediata se puede completar el pago directamente.
- V. Si la propiedad o inmueble no permite reserva inmediata se hace una solicitud de reserva. La solicitud de reserva es notificada al propietario quién puede confirmar o no la reserva. Si la confirmación es positiva, esto se le notifica al inquilino. Si la solicitud es denegada, el sistema devuelve el dinero al inquilino y es igualmente notificado al cliente.

Análisis

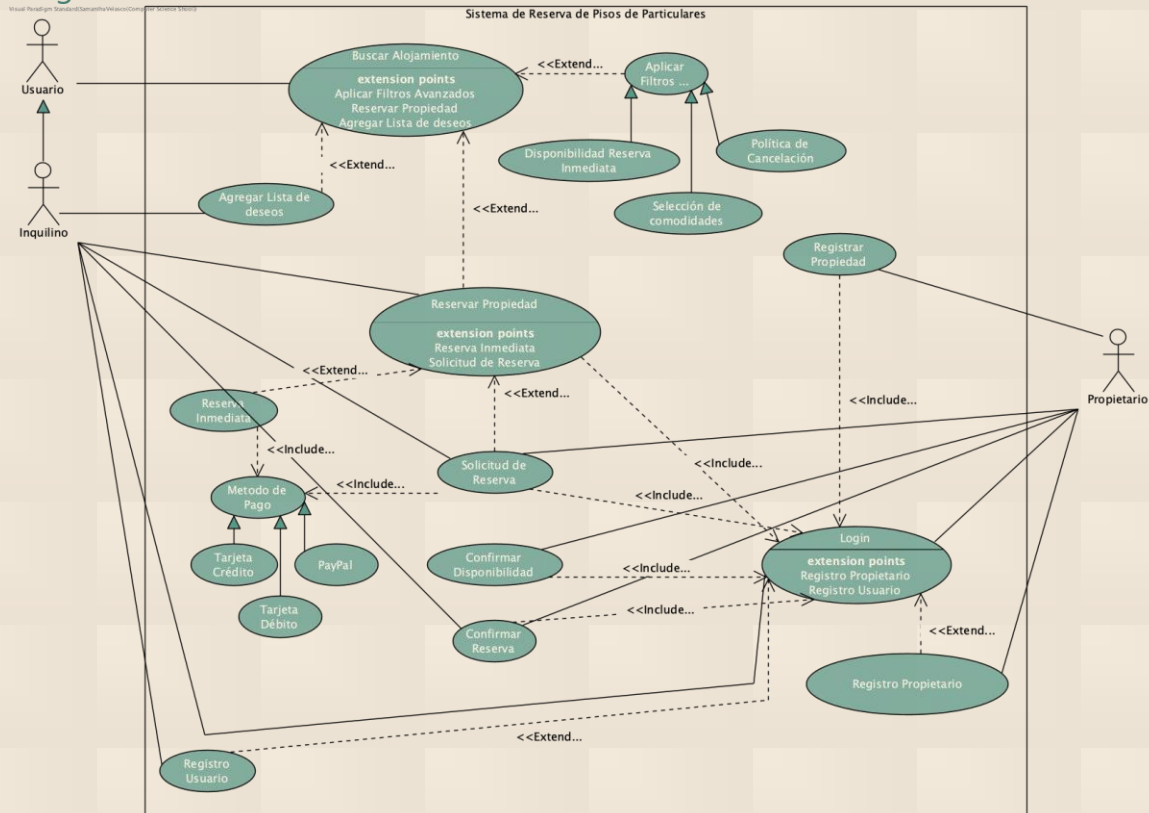
Actores

- I. Usuario. Representa a cualquier persona que accede al sistema, ya sea para consultar alojamientos o iniciar el proceso de registro y autenticación.
- II. Inquilino. Representa al usuario que desea reservar alojamientos. Este actor accede a las funcionalidades de búsqueda, filtrado, gestión de reservas y pago, siendo el principal responsable de iniciar el proceso de reserva de una propiedad.
- III. Propietario. Representa al usuario que ofrece alojamientos en la plataforma. Es responsable de registrar propiedades, gestionar su disponibilidad y atender las solicitudes de reserva realizadas por los inquilinos.

Casos de uso

- I. Registro y autenticación. El acceso al sistema requiere un proceso previo de registro y autenticación. Dependiendo del rol del usuario, el sistema permite el registro como inquilino o propietario, extendiendo el comportamiento básico de autenticación.
- II. Búsqueda. El sistema permite al inquilino realizar búsquedas de alojamientos, pudiendo aplicar filtros avanzados y seleccionar comodidades específicas. Estas funcionalidades se modelan mediante relaciones de extensión, ya que amplían el comportamiento básico de la búsqueda según las necesidades del usuario.
- III. Reserva. El proceso de reserva constituye uno de los casos de uso centrales del sistema. Este proceso puede realizarse mediante una reserva inmediata o a través de una solicitud de reserva, incluyendo siempre la confirmación de disponibilidad y la validación final de la reserva.
- IV. Pago. El sistema contempla diferentes métodos de pago, los cuales se incluyen como parte del proceso de reserva. Estas alternativas permiten al inquilino seleccionar el método más adecuado sin afectar al flujo general del sistema.

Diagrama



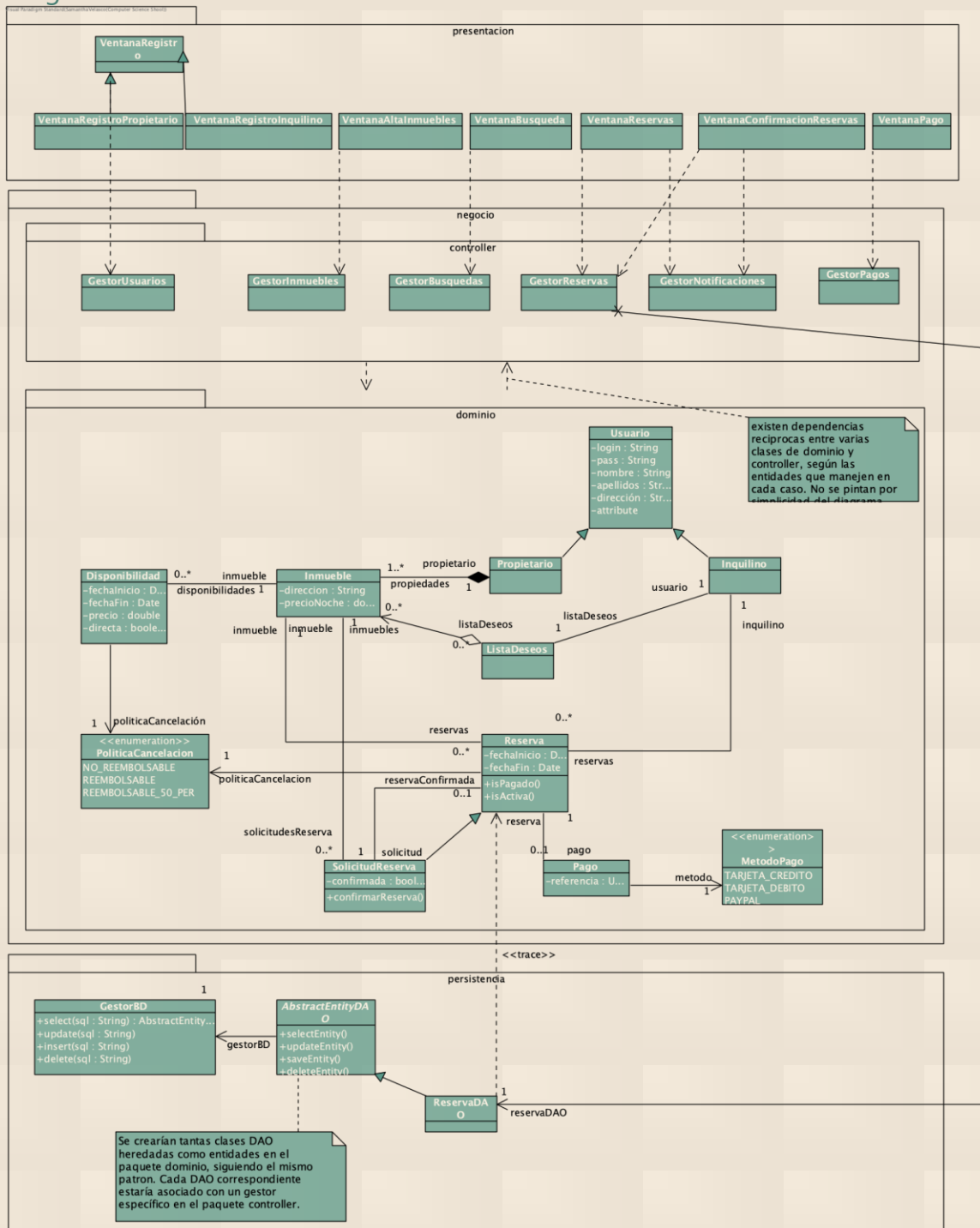
Diseño

Capas

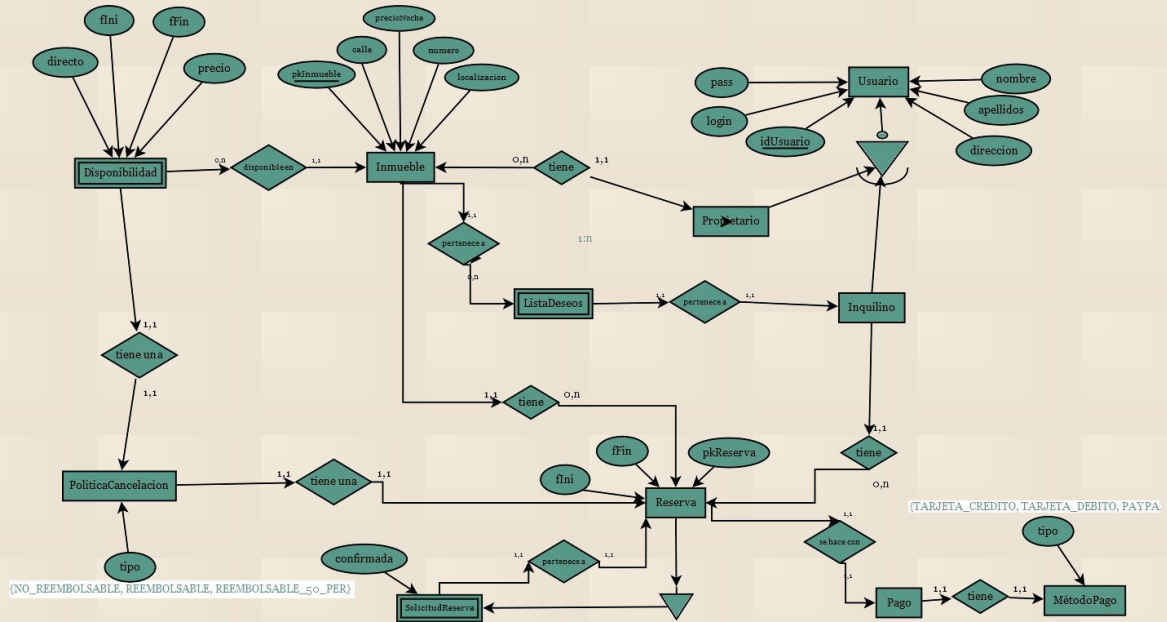
- I. **Presentación.** La capa de presentación agrupa las clases encargadas de la interacción con el usuario. Cada ventana representa una funcionalidad concreta del sistema y se corresponde directamente con los casos de uso definidos en la fase de análisis.
- II. **Negocio/Controlador.** La capa de negocio contiene los gestores responsables de coordinar la lógica del sistema. Estas clases actúan como intermediarias entre la presentación y el dominio, garantizando una separación clara de responsabilidades.
- III. **Dominio.** El dominio modela las entidades principales del sistema y sus relaciones, representando los conceptos del mundo real involucrados en la reserva de alojamientos entre particulares.
- IV. **Persistencia.** La capa de persistencia abstrae el acceso a la base de datos mediante el uso de objetos DAO, desacoplando el modelo de dominio de los detalles de almacenamiento.

Esta organización permite separar las responsabilidades, lo que facilita el mantenimiento y la comprensión del código. La capa de presentación se mantiene independiente, permitiendo sustituir la interfaz sin afectar al sistema. Por otro lado, la capa de persistencia está aislada, lo que facilita la transición a otras tecnologías en el futuro si fuese necesario.

Diagrama



Diagrama



Funcionalidades

Usuario

- I. Registro y autenticación de usuarios. Permite registrar a los inquilinos y propietarios, almacenando sus datos en la base de datos Derby. El sistema incluye inicio de sesión y validación de credenciales, asegurando el acceso a las funcionalidades según el tipo de usuario. Para crear un usuario, su *login* debe de ser un correo electrónico con @ y dominio, sino, no deja continuar con el registro de usuario.
- II. Inicio. Todos los usuarios aparecerán por defecto en la página de inicio, con su diseño único y una barra con enlaces a otras funciones de la aplicación. Cuando un usuario se autentique, dependiendo de si es inquilino o propietario, más funcionalidades serán mostradas arriba.

Propietario

- III. Alta de inmuebles. Los propietarios pueden dar de alta nuevos inmuebles mediante un formulario, introduciendo información como la ubicación, tipo de alojamiento, precio o si se necesita confirmar la reserva o si es automática. Cada inmueble queda asociado al propietario que lo registró.
- IV. Notificaciones. Cuando alguien hace una reserva de un inmueble que necesita confirmación, será mostrado en la página de inicio con un número dependiendo de cuantas solicitudes tenga. Este podrá aceptarlas o rechazarlas.

Inquilino

- V. Consulta de inmuebles. Los usuarios pueden consultar el listado completo de inmuebles disponibles en el sistema. La interfaz permite visualizar información relevante de cada propiedad. Se ha implementado una navegación fluida entre vistas, permitiendo pasar del listado a la ficha del inmueble o al formulario de reserva.
- VI. Reserva de inmuebles. Los usuarios que hayan iniciado sesión pueden reservar un inmueble seleccionado. El sistema registra la reserva en la base de datos después de haber realizado el pago, vinculando al usuario y al inmueble correspondiente.
- VII. Lista de deseos. Los inquilinos pueden añadir inmuebles a su lista de deseos para poder visualizarlos después sin tener que acceder a la búsqueda.
- VIII. Pago. Se mostrará al inquilino el precio total por las noches de su estancia y los tres métodos de pago de los que dispone la página para que elija con el que se sienta más cómodo.
- IX. Política de cancelación. No es algo que controlen los propietarios, dependiendo del número de días que el inquilino desee hospedarse, habrá una política u otra. Si son menos de 3 días, es completamente reembolsable. Si es entre 3 y 8 días, la política será del 50% reembolsable. Si son más de 8 días, no será reembolsado el dinero.
- X. Todas las reservas. Los inquilinos tendrán una página en la que podrán ver todas sus reservas, confirmadas, por confirmar, aceptadas y rechazadas.

Implementación

Requisitos previos

Herramienta	Versión recomendada	Descripción
Java JDK	21 o superior	Necesario para ejecutar <i>Spring Boot</i>
Maven	3.9	Sistema de gestión y compilación del proyecto
Git	Última versión	Para clonar y gestionar el repositorio
Eclipse IDE for Java Developers	2025-09 o superior	Entorno de desarrollo integrado
Spring Boot	3.x	Framework principal del proyecto
DBeaver	Última versión	Cliente SQL para gestionar la base de datos
PostgreSQL / MySQL	-	Motor de base de datos utilizado

Tecnologías utilizadas

- I. Eclipse. Es el entorno de desarrollo integrado (IDE) elegido para este proyecto porque ofrece una integración completa con **Java** y **Maven**, lo que facilita la compilación, ejecución del código. Además, permite gestionar el proyecto de forma estructurada y visual, y es uno de los recomendados por los profesores.
- II. Maven. Se utiliza como herramienta de gestión de dependencias y automatización de compilaciones. Gracias a Maven, es posible manejar las librerías necesarias (por ejemplo, *Spring Boot*, Derby o JPA) y garantizar que todos los miembros del equipo tengan el entorno de trabajo configurado de la misma forma, lo que mejora la coherencia del proyecto.
- III. Spring Boot. Es el *framework* principal del proyecto. Permite desarrollar aplicaciones Java estructuradas bajo el patrón MVC (Modelo–Vista–Controlador) y facilita la creación de servicios REST. Su uso reduce significativamente configuración manual y mejora la modularidad y escalabilidad del sistema.
- IV. Apache Derby. Se ha elegido como base de datos relacional embebida por su facilidad de uso y su integración directa con proyectos Java. Al ser ligera y no requerir instalación adicional, resulta ideal para entornos de desarrollo y pruebas, permitiendo que cada miembro del equipo trabaje con su propia base de datos local de manera sencilla. Se habían valorado el uso de MySQL, o H2 entre otras, pero nos hemos decantado por apache al final.
- V. DBeaver. Es una herramienta de administración de bases de datos que permite conectarse a Derby y visualizar sus tablas de forma gráfica. Se utiliza para realizar consultas SQL sin necesidad de hacerlo desde código, lo que facilita las tareas de desarrollo y depuración.

Estructura del proyecto

La estructura del proyecto sigue una arquitectura en capas, que separa: presentación, lógica de negocio y persistencia, junto con un paquete raíz encargado del arranque y configuración de la aplicación Spring Boot.

- I. Aplicación. Contiene la clase principal de arranque de Spring Boot. Incluye la configuración necesaria para la inicialización y despliegue de la aplicación. No forma parte de la lógica de negocio.
- II. Controladora. Gestiona las peticiones HTTP, recibe las solicitudes del cliente web, llama a la capa de servicio y devuelve la respuesta o vista correspondiente. No contiene lógica de negocio.
- III. Dominio. Contiene las entidades del sistema y el modelo de datos. Representa los conceptos principales del negocio. Puede incluir validaciones simples y relaciones ente entidades.
- IV. Servicio. Implementa la lógica de negocio del sistema. Coordina operaciones entre controladores y persistencia. Aplica reglas, validaciones y procesos del dominio.
- V. Persistencia. Gestiona el acceso a la base de datos. Contiene repositorios DAOs. Encapsula las operaciones CRUD y consultas.
- VI. Templates. Contiene las vistas HTML del sistema. Se usa con motores de plantillas como Thymeleaf. Permite generar contenido dinámico para el usuario.

Calidad del software

Análisis y Configuración de Reglas

Tras realizar una primera ejecución del análisis estático de código, el sistema detectó un total de 21 incidencias iniciales. La mayoría de estas incidencias estaban relacionadas con convenciones de documentación de paquetes. Para solucionar este bloque de incidencias, se tomó la decisión técnica de desactivar la regla *"Add a package-info.java file"*, basándose en los siguientes criterios:

- I. Redundancia: La arquitectura del proyecto ya define explícitamente la responsabilidad de cada paquete mediante su nomenclatura, por lo que un archivo de información extra es innecesario.
- II. Mantenibilidad: La regla obliga a mantener archivos vacíos o con información duplicada que no aporta valor real al desarrollo.

Nota sobre el entorno de pruebas: Durante esta fase, se detectó que no era posible ejecutar las pruebas directamente en SonarQube debido a una incompatibilidad de la herramienta JaCoCo. Esta aún no está actualizada para reconocer el formato bytecode (v69) generado por Java 25, impidiendo la lectura del código.

Inyección de Dependencias

Tras el primer filtrado, un segundo análisis arrojó 16 incidencias restantes. La principal alerta se refería a la inyección de dependencias en los campos (*Field dependency injection*). Se procedió a desactivar esta regla bajo la siguiente justificación:

- I. Aunque la inyección por constructor es teóricamente superior para facilitar test unitarios aislados, en este proyecto se ha priorizado el uso de la inyección de campo (*@Autowired*) para mantener la simplicidad y legibilidad del código en los controladores Spring MVC.

Refactorización y Corrección de "Code Smells"

Para solucionar las incidencias de mantenibilidad restantes, se llevaron a cabo las siguientes acciones de refactorización manual:

- II. Corrección en la Instanciación del Logger
 - a. Archivos afectados: `GestorInmuebles.java` y `LNListaDeseos.java`.
 - b. Incidencia: El objeto `Logger` se estaba inicializando con una clase incorrecta (copia de otro archivo), provocando que los *logs* del sistema aparecieran mal etiquetados y dificultando la trazabilidad.
 - c. Solución: Se actualizó el parámetro `LoggerFactory.getLogger(...)` para que coincidiera con la clase real (`GestorInmuebles.class` y `LNListaDeseos.class`), asegurando que los registros reflejen el origen correcto.

- III. Limpieza de Código (Dead Code e Imports)
 - a. Código Muerto: En el archivo `LNReservas.java`, se eliminaron bloques de código comentados que ya no eran operativos. SonarQube marca esto como *Code Smell* porque el historial debe gestionarse mediante Git, no con comentarios.
 - b. Importaciones no usadas: En `InmuebleDAO.java`, se eliminó la importación de `java.util.Optional` que no se estaba utilizando, limpiando así las dependencias de la clase.
- IV. Uso de Constantes
 - a. Se mejoró la mantenibilidad definiendo constantes para evitar "cadenas mágicas" repetidas. Por ejemplo, se creó la constante `USUARIO_AUTENTICADO` en `GestorUsuarios` para sustituir 5 repeticiones de la cadena de texto literal "usuarioAutenticado", facilitando futuros cambios en una sola línea.

Estado Final y Últimas Correcciones

Tras un tercer análisis, teníamos únicamente **2 incidencias** finales. Estas se resolvieron de la siguiente manera:

- I. Consistencia en Logs: Se realizó una corrección final en la inicialización del Logger en `GestorInmuebles.java` para garantizar la precisión de los registros.
- II. Limpieza en Tests: Se refactorizó `GestorInmueblesTest.java` eliminando los *matchers* `eq()` redundantes en las verificaciones de Mockito. Esto simplificó la sintaxis y mejoró la legibilidad de las pruebas unitarias sin alterar su lógica, alineándose con los estándares de *Clean Code*.

Pruebas

Este plan de pruebas se centra en la validación estructural de nuestro proyecto aplicando técnicas de Caja Blanca bajo el principio de la función MINI-MAX. Nuestro objetivo es lograr la máxima eficacia invirtiendo el mínimo tiempo y número de casos de prueba. Al tratarse de pruebas de Caja Blanca, se ha analizado la estructura interna del código para asegurar que se ejercitan los caminos lógicos de nuestro código.

- I. Cobertura de Decisiones. Se han diseñado casos para evaluar (true/false) cada condición lógica del gestor, asegurando que se recorren todas las ramas del flujo de control.
- II. Valores Límite. Para las condiciones numéricas críticas, no basta con probar el valor límite. Se ha aplicado la variante pesada, que exige probar:
 - a. El valor límite exacto.
 - b. El valor inmediatamente inferior al límite.
 - c. El valor inmediatamente superior al límite.

Gestor Inmuebles

Aunque seas un usuario que ha iniciado sesión, si no eres propietario, te salta un error:

Parámetros	Clase de equivalencia	Valores	Conjetura de errores
sesión. usuario	Usuario no autenticado (NULL) Usuario no propietario Usuario propietario	NULL Pepe Juan	Sesión nula Error Ok
Inmuebles. propietario	No existe (NULL) Existe	NULL Juan	Error Ok
Inmuebles. registrar	Fallo en el registro Éxito en el registro	NULL Guardado	Error Ok
Inmueble	Datos inválidos Datos válidos	precioNoche = -50 PrecioNoche = 100	Error Ok

ID	Metodo de prueba	session.usuario	Inmueble.propietario	Inmueble registrar	inmueble (input)	Expected Output
CP1	mostrarAlta	null				"alta", estaLogueado=false, altaPendiente=true
CP2	mostrarAlta	Pepe	null			"alta", error = "Solo los propietarios pueden dar de alta inmuebles"
CP3	mostrarAlta	Juan	Juan			"alta", esta Logueado=true
CP4	guardar	null	Pepe	New Inmueble		"alta", error="Debes iniciar sesión como propietario para registrar un inmueble."
CP5	guardar	Pepe	null	New Inmueble		"alta", error="Solo los propietarios pueden registrar alta inmuebles"
CP6	guardar	Juan	Juan	PrecioNoche=-50	null	"alta", error="Error al guardar:revise los datos del inmueble.."
CP7	guardar	Juan	Juan	PrecioNoche=100	Inmueble guardado	"alta", "Inmueble registrado correctamente."

LNBusqueda

Encapsula la lógica de negocio real, conteniendo múltiples estructuras de decisión condicionales para el filtrado de datos (Localización, Precio, Capacidad y Tipo).

Parámetros	Clase de equivalencia		Valores		Conjetura de errores
localización	NULL Coincidencia No coincidencia		NULL Madrid Tokio		Ignorar el filtro Filtrar Lista vacía
precioMax	Nulo <= 0 {0, infinito}		NULL o -1 100		Ignorar el filtro Filtrar
capacidad	Nulo <= 0 {0, infinito}		NULL o 0 4		Ignorar el filtro Filtrar
tipo	Todos NULL {Habitación individual} Tipo inexistente		“Todos” “Apartamento completo” “Iglú”		Ignorar el filtro Ok Lista vacía
ID	Localización	precioMax	Capacidad	Tipo	Expected output
CP1	NULL	NULL	NULL	Todos	Lista todos
CP2	Madrid	NULL	NULL	Todos	Lista pisos en madrid
CP3	Tokio	NULL	NULL	Todos	Lista vacía
CP4	NULL	-1	NULL	Todos	Lista todo ignora filtro
CP5	NULL	100	NULL	Todos	Lista todo tipo que tenga precio 100
CP6	NULL	49	NULL	Todos	Lista todo tipo que tenga precio 49
CP7	NULL	NULL	0	Todos	Lista todos
CP8	NULL	NULL	4	Todos	Lista todo tipo de inmueble con mas 4 capadidad
CP9	NULL	NULL	NULL	Apartamento completo	Lista de apartamentos Completos
CP10	NULL	NULL	NULL	Iglú	Lista vacía

Gestor Inicio

Parámetros		Clase de equivalencia	Valores		Conjetura de errores
sesión. usuario		NULL	NULL		Redirige al inicio Comprueba roles
		Existe	New Usuario Pepe		
Propietario		No propietario	NULL		Comprueba si es inquilino Asigna el rol
		Propietario	New Propietario()		
Inquilino		No inquilino	NULL		Usuario sin rol Asigna el rol de inquilino
		Inquilino	New Inquilino()		
ID	Método	session.usuario	Propietario	Inquilino	Expected output
CP1	Mostrar Inicio	NULL			esta logueado=false
CP2	Mostrar Inicio	usuario("Juan")	Objeto	null	EsPropietario = true
CP3	Mostrar Inicio	Usuario("Ana")	null	objeto	Es inquilino = true
CP4	Mostrar Inicio	Usuario ("Luis")	null	null	Es propietario = false Es inquilino = false

Gestor Lista de Deseos

En este caso funciona con -3 en el gestor porque la lógica la maneja el LNListaDeseos y es quien va a denegar esa agregación porque no puede existir un inmueble con id -3.

Parámetros	Clase de equivalencia		Valores	Conjetura de errores
sesión. usuario	NULL, Inquilino		NULL Juan (propietario)	Error Debes Iniciar sesión como Inquilino
	Propietario		Pepe (inquilino)	Acceso Concedido
IdInmueble	{0, infinito]		1 -3	OK OK (Se delega la responsabilidad a servicio)
	ID	Método	Session.usuario	Id Inmueble
CP1	agregar	NULL	NULL	Debes de iniciar sesión
CP2	agregar	Juan	NULL	Debes de ser inquilino
CP3	agregar	Pepe	NULL	Agrega inmueble 1 a la Lista de Deseos
CP4	agregar	Pepe	NULL	Agrega inmueble -3 a la lista de deseos

Gestor Notificaciones

Parámetros		Clase de equivalencia	Valores	Conjetura de errores
sesión. usuario		NULL, Autenticado	NULL Pepe	Ejecuta login Sigue la lógica
filtro(get)		{"TODOS", Pendiente", "Inventado"}	TODOS, Pendiente Inventado	Dao.FinbyPropietario Dao.FindbyPropietarioyEstado Excepcion
accion (post)		{"ACEPTAR", "RECHAZAR", "OTRO"}	ACEPTAR RECHAZAR OTRO	Llama aceptar solicitud Llama a rechazar Solicitud ignora
ID	Método	session.usuario	get	Expected output
CP1	Listar	NULL	TODAS	Llama findbyPropietario
CP2	Listar	Pepe	TODAS	Llama findbyPropietario y estado
CP3	Listar	Pepe	PENDIENTE	Llama findbyPropietario y estado
CP4	Listar	NULL	INVENTADO	Error
CP5	Procesar	NULL	ACEPTAR	Llama a: InSolicitud.aceptarSolicitud
CP6	Procesar	NULL	RECHAZAR	Llama a: InSolicitud.rechazarSolicitud
CP7	Procesar	NULL	INVENTADO	Error

Gestor Pagos

Parámetros	Clase de equivalencia	Valores	Conjetura de errores	
Reserva Actual	NULL	NULL	Redirige a reserva	
	Existe Reserva	New Reserva	Sigue el flujo	
Tipo de reserva	{Confirmacion, Inmediata}	Confirmacion, Inmediata	Crea Solicitud Guarda reserva +Cobra	
Tipos de pago	{Tarjeta_credito,Tarjeta Debito,PayPal}	Paypal, Tarjeta_Credito	Flujo normal	
ID	Método	Session.Reserva	Tipo Reserva	Expected OutPut
CP1	Mostrar Formulario	Null	Inmediata	Redirige a reserva
CP2	Procesar Pago	Existe	Confirmacion	Total a pagar : x
CP3	Procesar Pago	null	Inmediata	Redirige a reserva
CP4	Procesar Pago	Existe	Confirmacion	Llama a crear solicitud
CP5	Procesar Pago	Existe	Inmediata	Llama a registrar pago

Gestor Reservas

Parámetros		Clase de equivalencia	Valores	Valores límite	Conjetura de errores
Días Estancia		[1-3]	3 días	3,	Asigna Reembolsable
		[4-8]	5 días	4,	Asigna
		[8,infinito]	10 días	8	Reembolsable_50_por_per Asigna No_Reembolsable
ID	Método	Tipo Reserva	Expected OutPut		
CP1	Procesar	3	Reembolsable		
CP2	Procesar	5	Reembolsable_50_por_per		
CP3	Procesar	10	No_Reembolsable		

Gestor Usuarios

Parámetros		Clase de equivalencia	Valores	Conjetura de errores
sesión. usuario		Formato válido	"hola"	Error "Login debe ser un correo valido"
		Formato inválido	"hola@gmail.com"	Continua Flujo
Propietario		{Propietario, Inquilino}	Juan Pepe	Guarda Propietario Guarda Inquilino
ID	Método	Login	Tipo de usuario	Expected output
CP1	registro	hola	Propietario	Error "Login debe ser un correo valido"
CP2	registro	juan@gmail.com	Propietario	OK Guarda propietario
CP3	registro	pepe@gmail.com	Inquilino	Ok guarda Inquilino

InmueblesDAO

Se ha seleccionado la clase InmuebleDAO para la realización de pruebas debido a que encapsula la lógica de acceso a datos relacionada con la entidad Inmueble, definiendo consultas personalizadas mediante Spring Data JPA. Dado que estas consultas filtran resultados en función de distintos atributos (numéricos y textuales), es fundamental verificar su correcto comportamiento aplicando criterios sistemáticos de selección de valores. Esta justificación es válida para todas las clases DAO, por lo que sería redundante repetirla en cada caso.

Parámetros	Clases de Equivalencia	Valores Seleccionados	Valores Límite	Conjetura de Errores
Capacidad	Menor que el mínimo	1	3	El inmueble no debe devolverse si no alcanza la capacidad requerida
Capacidad	Igual al mínimo	3	3	El inmueble debe devolverse
Capacidad	Mayor que el mínimo	6	3	El inmueble debe devolverse
Precio Noche	Menor que el máximo	50.0	100.0	El inmueble debe devolverse
Precio Noche	Igual al máximo	100.0	100.0	El inmueble se devuelve
Precio Noche	Mayor que el límite	150.0	100.0	El inmueble no se devuelve
Tipo	Coincide	"Casa"	—	El inmueble aparece en el resultado
Tipo	No coincide	"Casa"	—	El inmueble no aparece
Localización	Contiene texto	"Toledo"	—	Coincidencia parcial ignorando mayúsculas
Localización	No contiene	"Toledo"	—	No debe aparecer
Propietario	Correcto	Propietario A	—	Solo se devuelven inmuebles del propietario indicado
Propietario	Incorrecto	Propietario A	—	No deben devolverse inmuebles

ID	Método	Capacidad	Precio Noche	Tipo	Localización	Propietario	Expected Output
CP1	findByCapacidad GreaterThanEqual	1, 3, 6	—	—	—	A	Devuelve inmuebles con capacidad ≥ 3
CP2	findByPrecioNoche LessThanEqual	—	50, 100, 150	—	—	—	Devuelve inmuebles con precio ≤ 100
CP3	findByTipo	—	—	Piso / Chalet	—	—	Solo devuelve inmuebles tipo seleccionado
CP4	findByLocalizaciónContainingIgnoreCase	—	—	—	"Madrid"	—	Devuelve inmuebles cuya localización contiene "Madrid"
CP5	findByPropietario	—	—	—	—	A	Devuelve solo inmuebles del propietario A

InquilinoDAO

Parámetros	Clases de equivalencia	Valores seleccionados	Valores Límite	Conjetura de errores
Login	Usuario Existente	"pepe@email.com"	—	Debe devolver el objeto Inquilino asociado.
Login	Usuario Inexistente	"noexiste@email.com"	—	Debe devolver null.
Login	Formato vacío/nulo	"" o null	—	Debe devolver null

ID	Método	Login (Input)	Expected Output
CP1	findByLogin	"pepe@email.com"	Devuelve el objeto Inquilino con login "pepe@email.com" .
CP2	findByLogin	"noexiste@email.com"	Devuelve null (la búsqueda no encuentra resultados).

PropietarioDAO

Parámetros	Clases de Equivalencia	Valores Seleccionados	Valores Límite	Conjetura de Errores
Login	Usuario Existente	"propietario@email.com"	—	Debe devolver la entidad propietario correcta.
Login	Usuario Inexistente	"nadie@email.com"	—	Debe devolver null.

ID	Método	Input (Login)	Expected Output
CP1	findByLogin	"propietario@email.com"	Devuelve objeto Propietario con ese login.
CP2	findByLogin	"nadie@email.com"	Devuelve null.

ReservaDAO

Parámetros	Clases de Equivalencia	Valores Seleccionados	Valores Límite	Conjetura de Errores
Inmueble	Mismo Inmueble	Inmueble A	—	Filtra correctamente por ID inmueble.
Inmueble	Otro Inmueble	Inmueble B	—	No debe devolver reservas de otros inmuebles aunque las fechas coincidan.
Fechas	Solapamiento Total/Parcial	[Inicio, Fin] dentro o cruzando rango existente	FechaFin solicitud == FechaInicio existente	Devuelve la reserva (colisión).
Fechas	Sin Solapamiento	Rango completamente fuera (antes o después)	FechaInicio solicitud > FechaFin existente	Lista vacía (Disponible).

Contexto para los casos: Existe una Reserva ya guardada en BD para el **Inmueble A** del **01/01/2024 al 10/01/2024**.

ID	Método	Inmueble (Input)	Fechas (Input)	Expected Output
CP1	findReservasSolapadas	Inmueble A	05/01/2024 - 15/01/2024	Lista size = 1 (Detecta solapamiento).
CP2	findReservasSolapadas	Inmueble A	11/01/2024 - 20/01/2024	Lista size = 0 (No hay solapamiento, fechas adyacentes pero distintas si la lógica lo permite, o solapada si toca el límite). <i>Nota: Según la query <= y >= tocan bordes.</i>
CP3	findReservasSolapadas	Inmueble A	20/01/2024 - 25/01/2024	Lista size = 0 (Rango totalmente fuera).
CP4	findReservasSolapadas	Inmueble B	05/01/2024 - 15/01/2024	Lista size = 0 (Fechas coinciden pero inmueble distinto).
CP5	findReservasPorInmueble	Inmueble A	N/A	Devuelve todas las reservas del Inmueble A.

SolicitudReservaDAO

Parámetros	Clases de Equivalencia	Valores Seleccionados	Conjetura de Errores
Inmueble	Inmueble con solicitudes	Inmueble A	Devuelve lista correcta.
Inmueble	Inmueble sin solicitudes	Inmueble B	Lista vacía.
Propietario	Propietario con solicitudes	Propietario A	Devuelve solicitudes de todos sus inmuebles (Join implícito).
Estado y Propietario	Coincide ambos	Prop A + PENDIENTE	Devuelve solicitud específica.
Estado y Propietario	Coincide Prop, No Estado	Prop A + ACEPTADA	No debe devolver la pendiente.
Estado y Propietario	No Coincide Prop	Prop B + PENDIENTE	No debe devolver solicitudes del Prop A.

Contexto:

- **Propietario A** tiene **Inmueble A**.
- **Propietario B** tiene **Inmueble B**.
- **Solicitud 1:** Inmueble A, Estado PENDIENTE.
- **Solicitud 2:** Inmueble A, Estado ACEPTADA.
- **Solicitud 3:** Inmueble B, Estado PENDIENTE.

ID	Método	Inputs	Expected Output
CP1	findByInmueble	Inmueble A	Lista size = 2 (Sol 1, Sol 2).
CP2	findByPropietario	Propietario A	Lista size = 2 (Sol 1, Sol 2).
CP3	findByPropietario	Propietario B	Lista size = 1 (Sol 3).
CP4	findPendientesByPropietario	Prop A, PENDIENTE	Lista size = 1 (Solo Sol 1).
CP5	findPendientesByPropietario	Prop A, ACEPTADA	Lista size = 1 (Solo Sol 2).
CP6	findByPropietarioAndEstado	Prop B, PENDIENTE	Lista size = 1 (Solo Sol 3).

UsuarioDAO

Parámetros	Clases de Equivalencia	Valores Seleccionados	Conjetura de Errores
Credenciales	Correctas (Login y Pass coinciden)	"user", "1234"	Debe devolver el objeto Usuario.
Credenciales	Password Incorrecta	"user", "mal"	Debe devolver null.
Credenciales	Usuario Inexistente	"otro", "1234"	Debe devolver null.

Contexto: Se persiste un usuario con login "juan" y contraseña "secreta".

ID	Método	Inputs (Login, Pass)	Expected Output
CP1	findByLoginAndPass	"juan", "secreta"	Devuelve el objeto Usuario (Login Exitoso).
CP2	findByLoginAndPass	"juan", "incorrecta"	Devuelve null (Fallo de autenticación).
CP3	findByLoginAndPass	"pedro", "secreta"	Devuelve null (Usuario no existe).

Guía de instalación y ejecución

Pasos previos

- I. Verificar versiones.
 - a. *java -version*
 - b. *mvn -version*
 - c. *git --version*

Configuración del entorno

- II. Clonar el repositorio con los siguientes comandos.
 - a. *git clone https://github.com/SASS-ISO-II/AV.git*
- III. Cambiar la ruta de la carpeta *database* en la interfaz de *dbeaver* de eclipse.

Ejecución

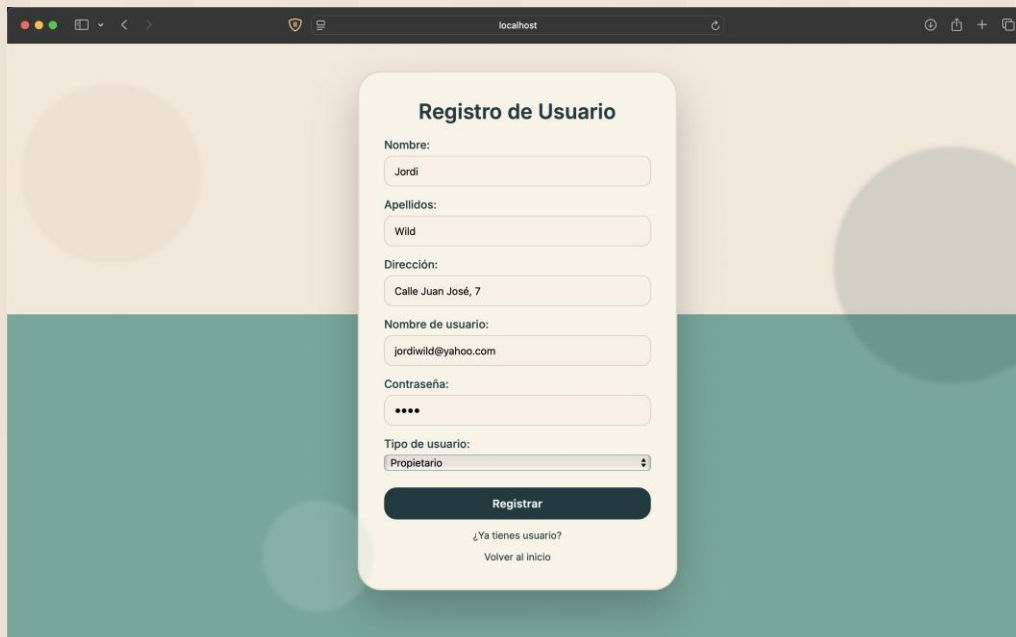
- I. Compilar el proyecto.
 - a. *mvn clean install*
- II. Ejecutar aplicación.
 - a. *mvn spring-boot:run*
- III. Acceder desde el navegador
 - a. <http://localhost:8081/inicio>

Demostración del sistema

La aplicación se ejecuta en esta url: <http://localhost:8081/>, lo cual, te lleva a la página de inicio.



En la parte superior, en un menú, puedes seleccionar varias acciones: registrarte, iniciar sesión, dar de alta un inmueble o ver los inmuebles.



Si vas al registro, te pedirá que insertes tus datos personales. En esta página no le puedes dar al botón de registrar sin haber rellenado todos los campos y sin que el nombre de usuario sea válido, tiene que ser un correo electrónico con @ y dominio. También hay enlaces a otras páginas relacionadas en todas las páginas para facilitar la navegación.

Iniciar Sesión

Nombre de usuario:

Contraseña:

Entrar

[¿No tienes cuenta? Regístrate](#)

[Volver al inicio](#)

En el inicio de sesión te pedirá tu correo electrónico y contraseña para poder entrar. Si el usuario existe, serás redirigido a tu página de usuario con tus datos dependiendo de si eres propietario o inquilino.

Propietario

Nombre: Jordi

Apellidos: Wild

Dirección: Calle Juan José, 7

Usuario: jordiwild@yahoo.com

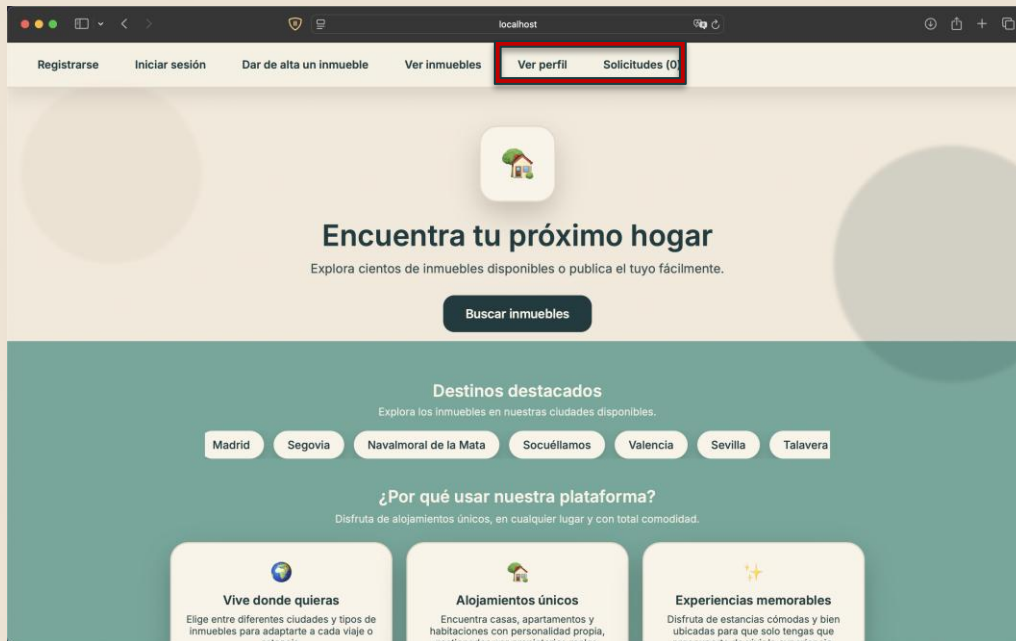
Dar de alta un inmueble

Entrar con otro usuario

Volver al inicio

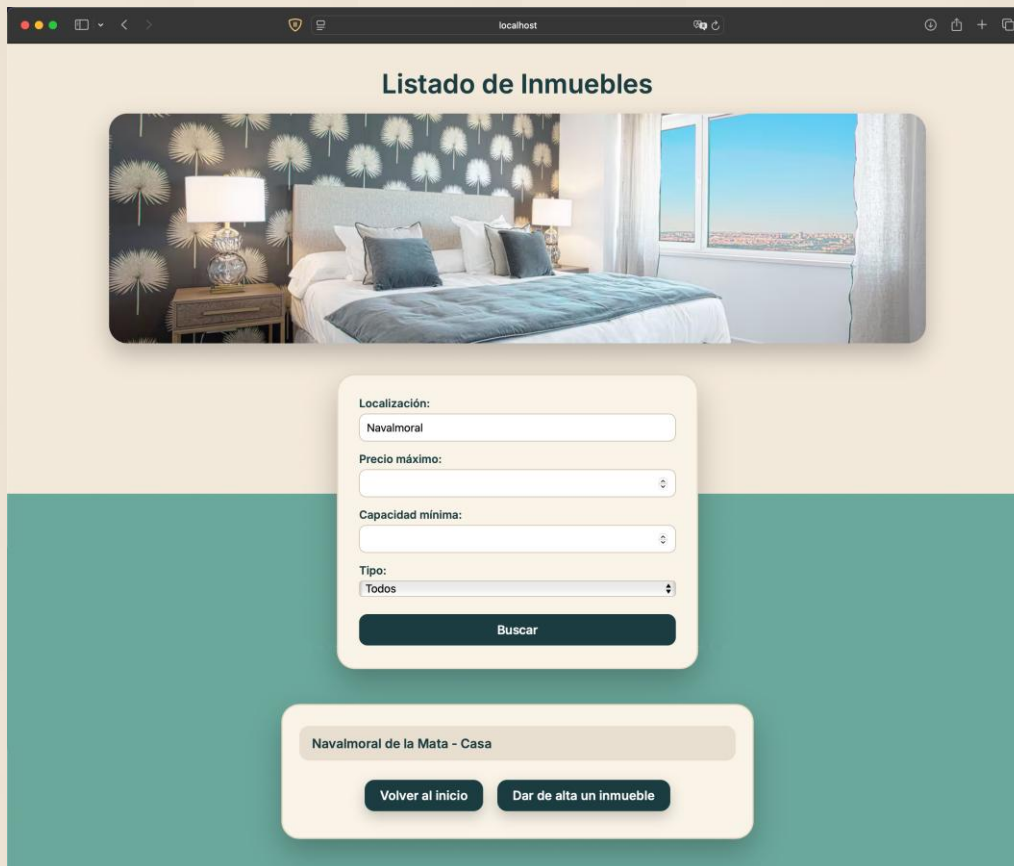
Ver notificaciones

Si has iniciado sesión y vas al inicio, te aparecerán una o dos pestañas nuevas dependiendo de si eres inquilino o propietario. Siendo cualquiera de los dos te aparecerá la de ver perfil y si eres propietario te aparecerá solicitudes también.

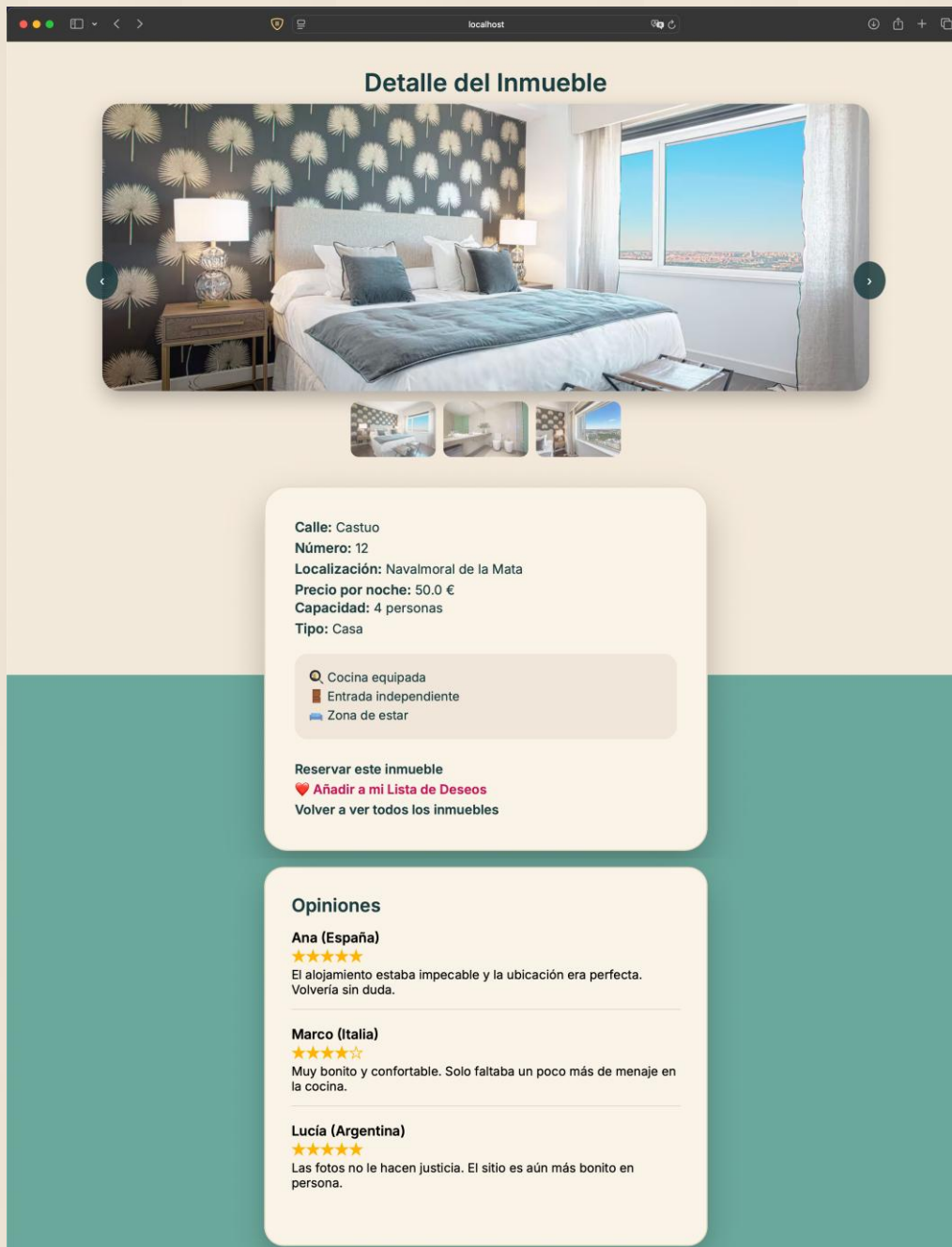


Siendo propietario, puedes dar de alta un inmueble y elegir qué tipo de reserva quieres que tenga el alojamiento.

Después de dar de alta al inmueble, este podrá ser encontrado en la página de inmuebles.



Y se podrá ver su página con sus detalles. Se han puesto imágenes predeterminadas para cada tipo de alojamiento y las mismas opiniones en todos los inmuebles ofertados, pero si se siguiese el desarrollo de esta aplicación, estaría bien implementarlo como funcionalidades reales.



Al darle al botón de reservar serás trasladado a la página de reservas, eliges los días que quieras y te saldrá la política de cancelación que tendrás dependiendo de cuántos días hayas elegido reservar. Si eliges días que ya están reservados, la página te dará un error y no te dejará continuar hasta que elijas días libres.

The screenshot shows a web browser window with the address bar set to 'localhost'. The page title is 'Reservar Inmueble'. The form is centered and contains the following fields and buttons:

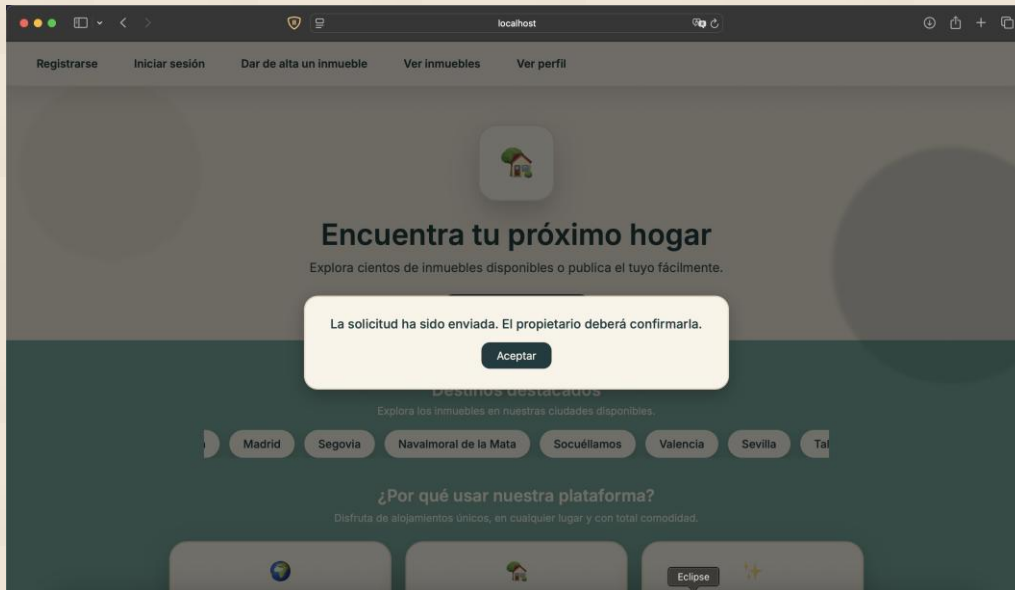
- Inmueble seleccionado:**
 - Calle: Castuo
 - Número: 12
 - Precio por noche: 50.0 €
- Fecha inicio:** 23/12/2025
- Fecha fin:** 25/12/2025
- Política de cancelación:** REEMBOLSABLE
- Fechas ya reservadas:**
- Buttons:** Confirmar Reserva, Reiniciar

Después te dará la información sobre tu reserva y tendrás que elegir un método de pago.

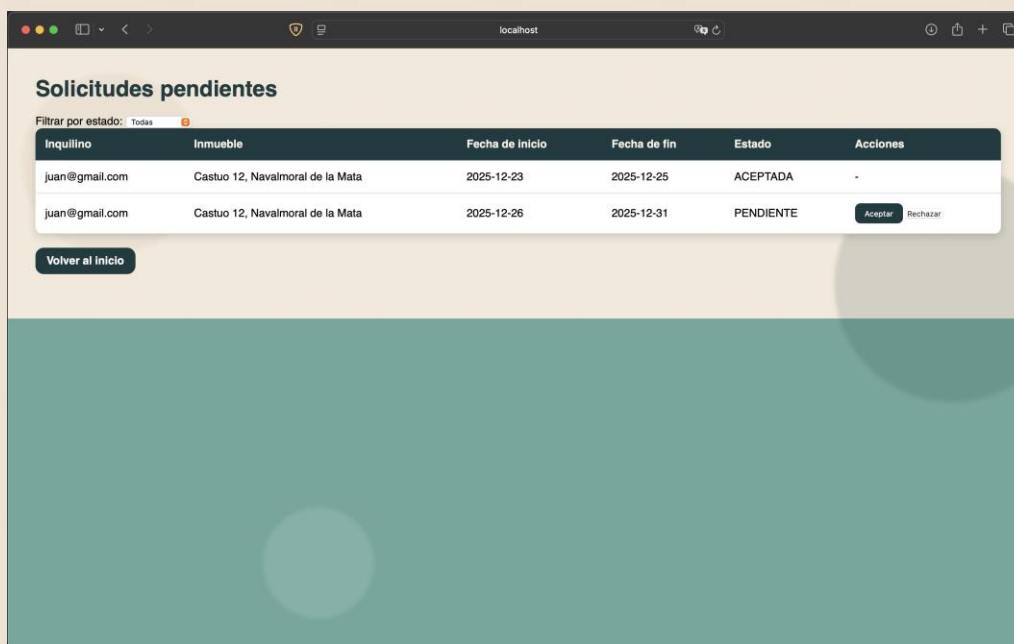
The screenshot shows a web browser window with the address bar set to 'localhost'. The page title is 'Confirmar Pago'. The form is centered and contains the following fields and buttons:

- Número de noches:** 2
- Total a pagar:** 100.0 €
- Método de pago:** TARJETA_CREDITO
- Buttons:** Realizar Pago, Volver al inicio

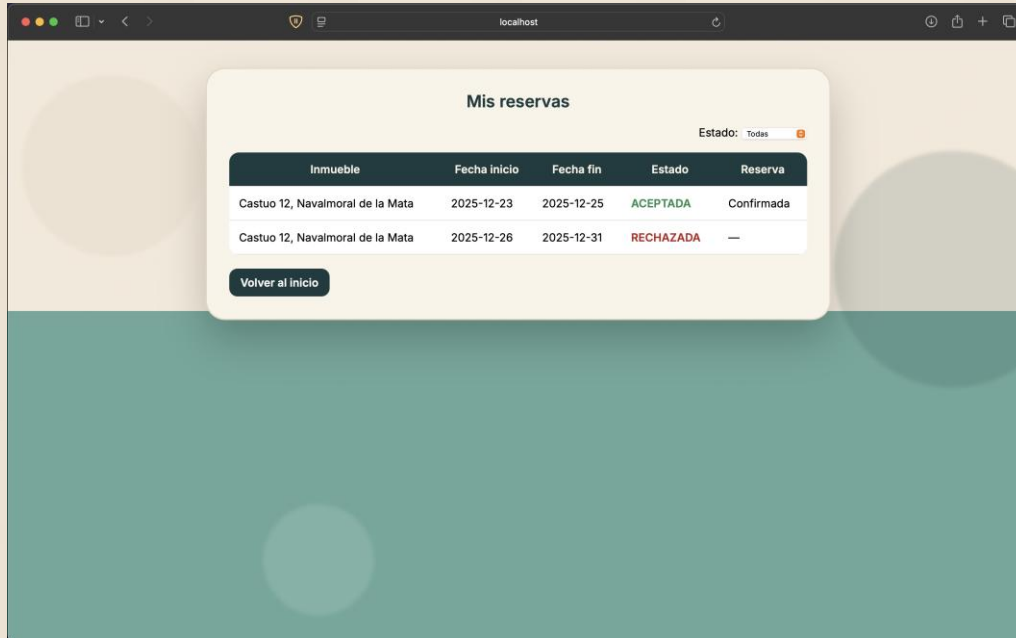
Al ser este un alojamiento con confirmación, el propietario tiene que confirmar o rechazar la solicitud. Si no tuviese confirmación, se podría ver como confirmada desde el apartado de tus solicitudes.



El propietario podrá aceptar o rechazar las solicitudes .



Se mostrarían así las reservas en la página del usuario solicitante.



Mantenimiento y evolución

Aunque el proyecto se da por finalizado tras su entrega y será archivado, a continuación, se describe un plan de mantenimiento hipotético, que recoge las actuaciones que se llevarían a cabo en caso de continuar con la evolución del sistema en un entorno real.

Mantenimiento correctivo

Ante la detección de errores o comportamientos no deseados, el mantenimiento correctivo se gestionaría mediante *GitHub Issues*, donde se registrarían los fallos detectados, su prioridad y el estado de resolución. Las correcciones se desarrollarían en la rama *hotfix* de encontrarse un error crítico en producción, y de no poder solventarse con rapidez se llevaría a *development*, asegurando que los cambios no afectan a la estabilidad de la rama principal. Antes de integrar los cambios:

- I. Se ejecutarían los tests unitarios con JUnit.
- II. Se validaría el resultado mediante *Surefire* en el proceso de construcción.
- III. Se analizaría la calidad del código con *SonarQube*, verificando el cumplimiento de los *Quality Gates* definidos.

Mantenimiento evolutivo

El mantenimiento evolutivo permitiría incorporar nuevas funcionalidades o mejorar las existentes, siguiendo una metodología ágil basada en iteraciones. Las nuevas características se planificarían como *Issues* y se desarrollarían en *ramas feature*, integrándose progresivamente en la rama de desarrollo. Cada evolución implicaría:

- I. Actualización del *backlog* en GitHub.
- II. Desarrollo incremental.
- III. Generación de nuevas releases versionadas.
- IV. Análisis continuo de calidad con SonarQube para prevenir deuda técnica.

Mantenimiento adaptativo

En caso de cambios en el entorno tecnológico, se realizaría un mantenimiento adaptativo orientado a garantizar la compatibilidad y estabilidad del sistema. Este tipo de mantenimiento incluiría:

- I. Actualización controlada de dependencias Maven.
- II. Ejecución completa del conjunto de pruebas automatizadas.
- III. Revisión del impacto mediante análisis estático del código.

Conclusiones

El desarrollo de este proyecto ha permitido aplicar de forma práctica los conocimientos adquiridos a lo largo de la asignatura, abordando el ciclo completo de vida del software desde la planificación y el diseño hasta la implementación, pruebas y control de calidad. Más allá del resultado funcional obtenido, el principal valor del trabajo reside en la comprensión de los procesos, herramientas y metodologías que intervienen en el desarrollo de un sistema software real.

Desde el punto de vista formativo, este trabajo ha contribuido a desarrollar competencias clave como la planificación del desarrollo, la autogestión del equipo, la resolución de problemas técnicos y la toma de decisiones justificadas. También ha puesto de relieve la importancia de la documentación como elemento fundamental para la comprensión, mantenimiento y evolución de un sistema software.

En conclusión, el proyecto ha cumplido los objetivos planteados a nivel académico, proporcionando una visión integral del proceso de desarrollo de software y sentando una base sólida para afrontar proyectos de mayor complejidad en un contexto profesional.